LoRA, LoRA plus embeddings, or full-parameter continued pretraining for Gemma 4 E4B: throughput on A100 and H100, and the cost structure of the first run

PinoAI engineering note · ENG-003 · technical report

Stan Lee V. Natividad
Kooya PH AI Research
stan@kooya.ph

2026-09-30

Abstract

The first continued-pretraining (CPT) run of the program will adapt Gemma 4 E4B on $T = 2 \times 10^9$ Filipino tokens. Two questions had to be answered before spending on it: which parameters to train, and what the run costs end to end. This note answers the **measurable half** of the first question (does each training method fit on one GPU, and how fast does it run) and all of the second. It does **not** say which method teaches Filipino best. That is the job of a separate pilot, in which all three methods train on real data.

Three arms were registered before any measurement: **A1**, LoRA r = 64, $\alpha = 128$ on all linear layers; **A2**, A1 plus fully trainable input embeddings, per-layer embedding tables and output head; **A3**, full-parameter training with torchao's 8-bit AdamW and bf16 stochastic rounding. Each arm searched gradient checkpointing {off, on} × micro-batch {1, 2, 4, 8} on one A100 80GB, and A1 and A3 were repeated on one H100 80GB. The main results:

1. **Full-parameter training is not slower than LoRA on this model.** On the H100, A1′s fastest setting gives 4,953 tokens/s and A3′s gives 4,755 (LoRA 4% faster, inside the ~4% run-to-run spread). At *equal* settings (micro-batch 4, checkpointing on) A3 is 10% *faster* than A1 (4,755 versus 4,315). LoRA's win at the selected settings comes from memory, not compute: its small optimizer state lets it drop checkpointing at micro-batch 1. On the A100-SXM4 the order flips at the selected settings: A3 runs at 2,002 tokens/s and A1 at 1,914 (full 4.6% faster).
2. **Full-parameter memory is what the arithmetic predicts.** A3 at micro-batch 1 peaks at 44.4 GiB, which equals 7.94B parameters × 6 bytes (bf16 weights and gradients, two 8-bit optimizer moments) to within 0.05 GiB.
3. **"Train the embeddings" is not small on Gemma 4 E4B.** Its per-layer embedding tables make A2 a 4.36B-parameter change. The adapter library copies the tables, the reported total grows from 8.14B to 12.30B, the tied input and output embeddings become untied, and A2 is the slowest and most memory-hungry arm.
4. **The H100 is cheaper per token.** At list price it costs 58% more per hour but runs A1 2.59× and A3 2.37× faster than the A100-SXM4, so cost per token falls by 39% (A1) and 33% (A3).
5. **All-in cost.** Plumbing test, three-arm pilot, main run, evaluation, checkpoint storage and a 25% retry buffer fit within the approved amount with LoRA as the main arm and with full-parameter training, both on the H100. The full-parameter plan's 1.0% margin is smaller than the measurement noise, and the retry buffer is what covers it.

All registered A100 rows are on the same variant, `A100-SXM4-80GB`. A1′s first A100 run landed on the PCIe variant. Under the project's registered hardware rule, it was re-measured on SXM4, and the PCIe run is kept only as a sensitivity row. At A1′s selected setting PCIe was 3.3% slower, and with checkpointing on it was 5.8–6.8% slower.

1. Registered design

The design was written down and registered in the project's decision log (DEC-001, "Gate F method v4 and training arms") before any v4 number existed. The probe and loss are the ones described in ENG-001: sequence length 4,096, bf16, synthetic uniform-random tokens, 5 warm-up and 20 timed steps, a chunked cross-entropy loss (method v2) whose equality with the Hugging Face loss is checked in every attempt before timing, and one fresh process per attempt. The container image is ENG-001′s fast-kernel image (v3) with torchao 0.18.0 added.

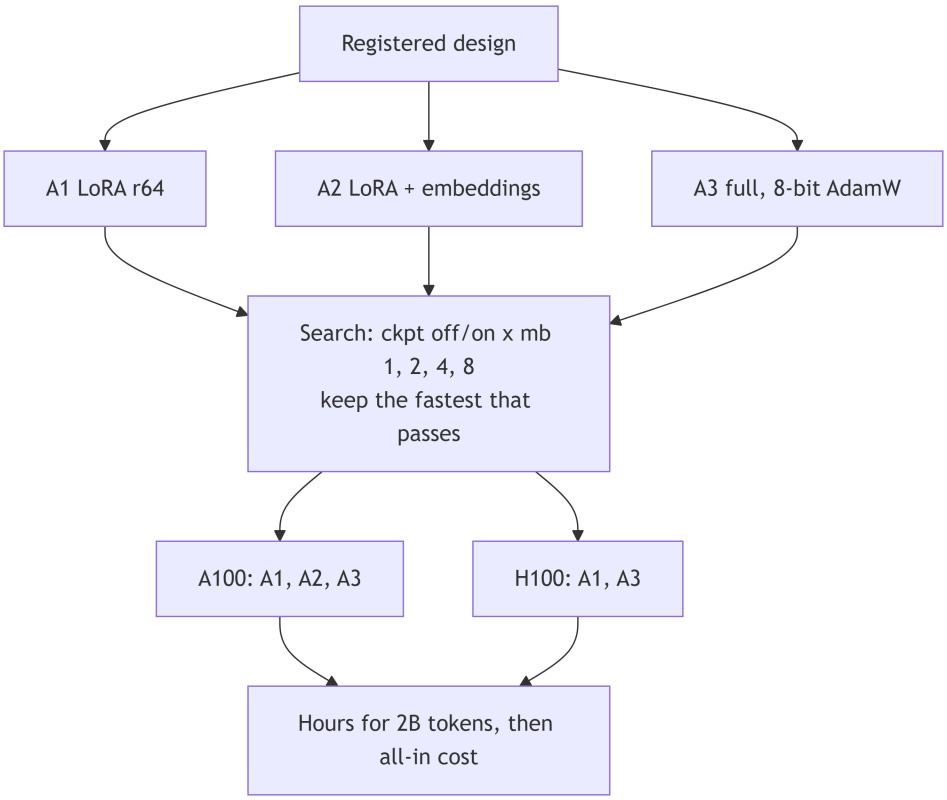
Table 1: The three registered arms. A1 is also a same-day control against ENG-001's LoRA numbers.

arm	what trains	optimizer (probe)
A1 LoRA	LoRA r = 64, α = 128, dropout 0, all linear layers	`torch.optim.AdamW`, lr 1e-4
A2 LoRA+emb	A1 plus full copies of `embed_tokens`, `embed_tokens_per_layer`, `lm_head` (`modules_to_save`)	`torch.optim.AdamW`, lr 1e-4
A3 full	every parameter	torchao `AdamW8bit`, `bf16_stochastic_round=True`, lr 1e-5

Update precision for A3. Pure-bf16 weights with an ordinary optimizer round away most updates of size ~1e-5, since bf16 has only 8 bits of mantissa. The registration therefore required A3 to declare an update-precision scheme (fp32 master weights, or a Kahan-summation or stochastic-rounding optimizer), and made pure-bf16 updates ineligible. A3 uses stochastic rounding, which costs almost no extra memory. The measured memory is thus the memory of the configuration that would really train.

Search and selection. For each micro-batch in 1, 2, 4, 8 the probe first tries checkpointing off and, only if that runs out of memory (OOM), checkpointing on. The search stops at the first micro-batch where both fail. The selected setting is the one with the highest tokens/s that fits and passes the loss check. A1 and A3 were repeated on one H100 80GB with the same search.

All-in cost, declared in advance. Compute for every planned stage (plumbing test, pilot head-to-head of the three arms, main run, evaluation) at the provider's list price, plus checkpoint storage (size × count × retention × $/GiB-month), plus a 25% retry buffer on compute that may not be tuned later.



2. Results

2.1 Selected settings

Table 2: Each arm at its selected setting (fastest that fits and passes the loss check). Hours are T / tok/s / 3,600 with T = 2 × 10⁹, a lower bound on wall time (no data loading, evaluation or checkpoint I/O).

arm	device (as recorded)	selected setting	tok/s	peak GiB	h for 2B tokens
A1 LoRA	A100-SXM4-80GB	mb 1, ckpt off	1,913.7	62.55	290.3
A1 LoRA	A100 80GB PCIe (sensitivity row)	mb 1, ckpt off	1,850.1	62.55	300.3
A1 LoRA	H100 80GB HBM3	mb 1, ckpt off	4,952.9	62.60	112.2
A2 LoRA+emb	A100-SXM4-80GB	mb 4, ckpt on	1,773.4	62.11	313.3
A3 full	A100-SXM4-80GB	mb 4, ckpt on	2,001.9	51.73	277.5
A3 full	H100 80GB HBM3	mb 4, ckpt on	4,754.6	51.78	116.8
A2 LoRA+emb	H100 80GB HBM3	not measured	-	-	-

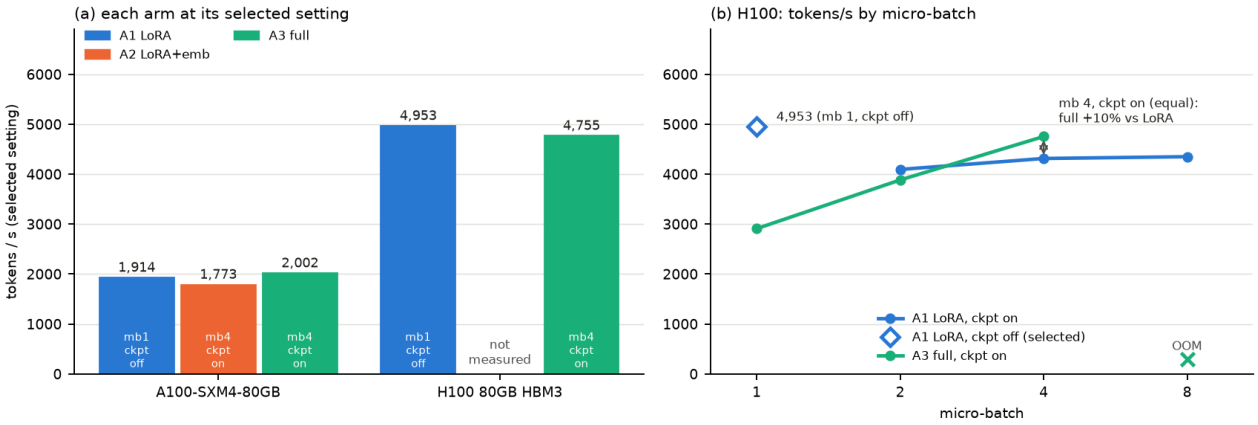


Figure 1: (a) Tokens/s of each arm at its own selected setting, by GPU (all A100 rows on A100-SXM4-80GB; the PCIe sensitivity row is in Table 2 and Table 9). (b) On the H100, tokens/s by micro-batch with checkpointing on for A1 and A3 (equal settings), and A1's selected checkpointing-off point.

2.2 Full search grid

Table 3: Every attempt: tokens/s with peak allocated GiB in parentheses. Bold marks the selected cell. "-" means not tried: checkpointing on is tried only when off ran out of memory at that micro-batch, and the search stops after a micro-batch where both fail.

arm · GPU · checkpointing	mb 1	mb 2	mb 4	mb 8
A1 · A100 SXM4 · ckpt off	**1,914 (62.5)**	OOM	OOM	OOM
A1 · A100 SXM4 · ckpt on	-	1,678 (27.2)	1,777 (37.6)	1,823 (58.4)
A1 · A100 PCIe · ckpt off	**1,850 (62.5)**	OOM	OOM	OOM
A1 · A100 PCIe · ckpt on	-	1,582 (27.2)	1,660 (37.6)	1,700 (58.4)
A1 · H100 · ckpt off	**4,953 (62.6)**	OOM	OOM	OOM
A1 · H100 · ckpt on	-	4,095 (27.2)	4,315 (37.6)	4,351 (58.5)
A2 · A100 SXM4 · ckpt off	OOM	OOM	OOM	OOM
A2 · A100 SXM4 · ckpt on	1,466 (56.6)	1,661 (56.7)	**1,773 (62.1)**	OOM
A3 · A100 SXM4 · ckpt off	OOM	OOM	OOM	OOM
A3 · A100 SXM4 · ckpt on	1,316 (44.4)	1,707 (44.3)	**2,002 (51.7)**	OOM
A3 · H100 · ckpt off	OOM	OOM	OOM	OOM
A3 · H100 · ckpt on	2,910 (44.4)	3,885 (44.3)	**4,755 (51.8)**	OOM

Checkpointing off fit only once per GPU, for A1 at micro-batch 1. Every other checkpointing-off attempt ran out of memory, including every A3 attempt. With checkpointing on, A2 and A3 both failed at micro-batch 8 on an allocation of 8.00 and 7.99 GiB respectively.

2.3 Parameters

Table 4: Parameter counts as reported by the probe after the adapter is applied. The "total" of A1 and A2 includes the adapter parameters and, for A2, the full trainable copies of the embedding tables and output head. The base model's configuration ties the input embedding and output head.

arm	trainable	total (as reported)	trainable share	modules_to_save	embed / lm_head tied after wrap
A1 LoRA	201,998,336	8,143,099,168	2.5%	none	n/a (base frozen, tied)
A2 LoRA+emb	4,362,747,904	12,303,848,736	35.5%	`embed_tokens`, `embed_tokens_per_layer`, `lm_head`	no (untied)
A3 full	7,941,100,832	7,941,100,832	100.0%	none	n/a (all trainable)

The counts reconcile exactly. A1′s total minus its trainable LoRA parameters equals A3′s total (the base model, 7,941,100,832 parameters). A2′s total exceeds A1′s by 4,160,749,568, and so does its trainable count: that number is the size of the copied `embed_tokens`, `embed_tokens_per_layer` and `lm_head` modules.

3. LoRA versus full-parameter on the same GPU

Both GPUs now give a same-variant comparison: A1 and A3 ran on the same device type on the H100 and on the A100-SXM4.

Table 5: H100 80GB HBM3, A1 versus A3. The first row compares each arm at its own selected setting; the others hold micro-batch and checkpointing equal. The last column is A3′s tokens/s relative to A1′s.

setting (H100 80GB HBM3)	A1 LoRA tok/s	A3 full tok/s	A3 vs A1
each arm's selected setting	mb 1 ckpt off: 4,953	mb 4 ckpt on: 4,755	−4.0% (LoRA faster)
equal: mb 1, ckpt on	not run (ckpt off fit)	2,910	-
equal: mb 2, ckpt on	4,095	3,885	−5.1% (LoRA faster)
equal: mb 4, ckpt on	4,315	4,755	+10.2% (full faster)
equal: mb 8, ckpt on	4,351	OOM	-

At the selected settings, LoRA is 4% faster (4,953 versus 4,755 tokens/s). That margin is about the size of the run-to-run spread measured in ENG-001, so the two are close to a tie. At equal settings, micro-batch 4 with checkpointing on, full-parameter training is **10% faster** (4,755 versus 4,315).

The reading we take from this: on Gemma 4 E4B, the rank-64 adapters on every linear layer, which run as separate small matmuls next to each frozen layer, cost more per token than full-parameter training's extra work of computing weight gradients. LoRA reaches its best number only because its optimizer state is small enough that checkpointing can be turned off at micro-batch 1, which removes the recomputed forward pass (+13.8% over A1′s best checkpointed setting). The advantage comes from memory, not from cheaper compute.

This reading holds at micro-batch 4 but not at every micro-batch. At micro-batch 2, A1 is 5% faster than A3 (4,095 versus 3,885). A3′s throughput rises steeply with micro-batch (+63% from 1 to 4), and A1′s checkpointed throughput is almost flat (+6% from 2 to 8). That pattern fits a large fixed cost per optimizer step for A3, updating and requantizing 7.94B parameters, spread over more tokens as the micro-batch grows. We did not profile the step, so this is an interpretation, not a measurement.

Table 6: A100-SXM4-80GB, A1 versus A3, laid out as Table 5.

setting (A100-SXM4-80GB)	A1 LoRA tok/s	A3 full tok/s	A3 vs A1
each arm's selected setting	mb 1 ckpt off: 1,914	mb 4 ckpt on: 2,002	+4.6% (full faster)
equal: mb 1, ckpt on	not run (ckpt off fit)	1,316	-
equal: mb 2, ckpt on	1,678	1,707	+1.7% (full faster)
equal: mb 4, ckpt on	1,777	2,002	+12.7% (full faster)
equal: mb 8, ckpt on	1,823	OOM	-

On the A100-SXM4 the picture points the same way, and more strongly. At the selected settings, full-parameter training is 4.6% faster than LoRA (2,002 versus 1,914 tokens/s). On this GPU, turning checkpointing off gains A1 only 5.0% over its best checkpointed setting, compared with 13.8% on the H100. At equal settings A3 is 12.7% faster at micro-batch 4 and 1.7% faster at micro-batch 2, so the micro-batch 2 reversal seen on the H100 does not appear on the A100. What holds on both GPUs: at the setting each arm would actually use, full-parameter training is within about 5% of LoRA, faster on one GPU and slower on the other, and cost per token does not favor LoRA on this model.

4. Memory

4.1 Full-parameter: 6 bytes per parameter

Table 7: Predicted persistent memory of A3 from its parameter count, against the measured peak at micro-batch 1 with checkpointing on.

component	bytes / parameter	GiB
bf16 weights	2	14.79
bf16 gradients	2	14.79
8-bit AdamW first moment	1	7.40
8-bit AdamW second moment	1	7.40
sum (7.94B × 6 bytes)	**6**	**44.37**
measured peak, A3 mb 1 ckpt on (A100)		44.36
measured peak, A3 mb 1 ckpt on (H100)		44.40

With bf16 weights (2 bytes), bf16 gradients (2 bytes) and two 8-bit AdamW moments (1 byte each), 7,941,100,832 parameters × 6 bytes = 44.37 GiB. The measured peaks at micro-batch 1 are 44.36 GiB (A100) and 44.40 GiB (H100). At micro-batch 1 and 2, A3's peak is therefore almost entirely model state, and activations under checkpointing add little. Even at micro-batch 4 (51.7 GiB) A3 uses less memory than A1 without checkpointing (62.6 GiB).

The match also shows the stochastic-rounding option adds no measurable state. The 8-bit optimizer's per-block scales are not in the 6-byte count. They are small, and the agreement to 0.05 GiB leaves no room for anything larger.

4.2 LoRA plus embeddings: the copies are the cost

Table 8: A1 versus A2 on the A100-SXM4 with checkpointing on (the same device string for both). Losses are the first and last of the 25 steps, on synthetic tokens.

mb (ckpt on)	A1 peak GiB	A2 peak GiB	A2 – A1 GiB	A1 loss first → last	A2 loss first → last
1	-	56.65	-	-	12.51 → 4.52
2	27.17	56.66	+29.49	13.43 → 12.36	12.66 → 6.15
4	37.59	62.11	+24.52	13.29 → 12.52	12.23 → 7.58
8	58.43	OOM	-	13.33 → 12.57	-

`modules_to_save` does not make the existing embedding tables trainable in place. It keeps the frozen originals and adds trainable copies of `embed_tokens`, `embed_tokens_per_layer` and `lm_head`, 4.16B parameters in all. Gemma 4 E4B's per-layer embedding tables are most of that. The copies need bf16 weights, bf16 gradients and two AdamW moments (stored in the parameter dtype by `torch.optim.AdamW`), about 8 bytes per parameter or 31.0 GiB. The measured A2 – A1 difference at equal settings is 24.5–29.5 GiB. That is the right size, and the exact figure depends on when in the step each peak occurs. The result is that A2 cannot turn checkpointing off at any micro-batch, and it runs 11% slower than A3 on the same SXM4 A100 (1,773 versus 2,002 tokens/s), while training about half as many parameters.

The copies also break weight tying. The base configuration has `tie_word_embeddings = true`, but after wrapping, the trained input-embedding and output-head copies do not share storage (`embed_lm_head_tied_after_wrap = false`). A real A2 run would train two separate matrices where the base model has one. Whether that matters for quality is for the pilot to measure.

The fast-falling A2 loss is not learning. A2′s loss falls from 12.23 to 7.58 over 25 steps at its selected setting, against about 13.3–13.4 ⟶ 11.8–12.0 for A1 and A3 at theirs. The probe draws one batch of uniform random ids (seed 0, ids below V − 1,024 with V = 262,144) and reuses it at every step. A uniform guess over those ids scores ln(261,120) = 12.47, so any loss below that comes from memorizing that one fixed batch. Trainable per-token lookup tables (the input and per-layer embeddings) and a trainable output head, at lr 1e-4, are well suited to that kind of fitting. A2′s steeper curve is the embeddings fitting the probe's synthetic batch, not learning Filipino, and it says nothing about how the arms will compare on real text. The learning rates also differ by arm (1e-4 for A1 and A2, 1e-5 for A3), so the loss columns cannot be compared between arms.

Disclosed crash. The first A2 run (`v4-a2.run1-crash-tiecheck.json`) exited with status 1 at micro-batch 1 for both checkpointing settings, before any measurement. The cause was a bug in the probe's new tie-check code, not memory; the file's automatic note "no micro-batch fit in memory" is therefore wrong. The check was fixed and A2 was re-run. The failed file is kept.

5. The A100 variant: PCIe sensitivity row

The provider's `A100-80GB` request can land on either the SXM4 (400 W) or the PCIe (300 W) variant, and the variant cannot be selected. A1′s first A100 run recorded `NVIDIA A100 80GB PCIe`, while A2 and A3 recorded `NVIDIA A100-SXM4-80GB`. The registered rule (DEC-001, "Gate F hardware variant") makes a PCIe number a sensitivity row only and requires the model to be re-measured on SXM4. That re-measure is the registered A1-A100 row in this note (`v4-a1.json`, 1,913.7 tokens/s, 290.3 h for 2B tokens). The PCIe run (`v4-a1.pcie-1.json`, 1,850.1 tokens/s, 300.3 h) is kept as the sensitivity row. Every A100 comparison between arms in this note is therefore within one variant.

Table 9: A1 on the A100, same method and image: the registered SXM4 run against the PCIe sensitivity run, at every setting both measured. ENG-001′s v3 run (same fast-kernel image, SXM4, checkpointing on) is shown as an earlier reference.

setting	ENG-001 v3, SXM4 (ckpt on)	v4 A1, SXM4	v4 A1, PCIe	PCIe vs SXM4
mb 1, ckpt off	-	1,913.7	1,850.1	−3.3%
mb 2, ckpt on	1,674	1,678.5	1,581.6	−5.8%
mb 4, ckpt on	1,766	1,776.6	1,659.9	−6.6%
mb 8, ckpt on	1,810	1,823.3	1,700.2	−6.8%

PCIe is 3.3% slower at A1′s selected setting (micro-batch 1, checkpointing off) and 5.8–6.8% slower with checkpointing on. The selected-setting gap is inside the ~4% run-to-run spread. The checkpointed gaps are outside it and fit the PCIe card's lower power limit. The SXM4 re-measure also reproduces ENG-001′s v3 SXM4 numbers to within 0.7% at every checkpointed micro-batch, even though the two runs are on different days and in different containers. That is a second repeat of the probe, and it is tighter than the 4.4% of ENG-001′s only earlier repeat.

The variant does not change the plan. A1 is cheapest on the H100 either way, since at $2.50/h an A100 would need about 3,135 tokens/s to match the H100′s cost per token.

6. Cost

6.1 Cost of the main run per arm and GPU

Table 10: Cost of the 2B-token main run alone, at list price (A100 80GB $2.50/h, H100 $3.95/h, read from the provider on 2026-09-30).

arm	GPU	tok/s	GPU-h for 2B	list $/GPU-h	$ for 2B tokens
A1 LoRA	A100	1,913.7	290.3	$2.50	$725.8
A1 LoRA	H100	4,952.9	112.2	$3.95	$443.1
A2 LoRA+emb	A100	1,773.4	313.3	$2.50	$783.2
A3 full	A100	2,001.9	277.5	$2.50	$693.8
A3 full	H100	4,754.6	116.8	$3.95	$461.5

The H100 costs 58% more per hour and runs A1 2.59× and A3 2.37× faster than the A100-SXM4, so the main run is 39% (A1) and 33% (A3) cheaper on it. Both ratios compare SXM4 against H100. Against the PCIe row, A1′s saving would read 41%. On the A100, full-parameter training is the cheaper main run ($693.8 versus $725.8 for LoRA). On the H100 the order is reversed ($443.1 versus $461.5).

6.2 All-in cost

Both H100 plans fit the approved amount for the whole plan, which was approved for compute alone: LoRA with a 6.2% margin, full-parameter with 1.0%. The full-parameter margin is smaller than the ~4% throughput spread, so a slower main run could push that plan slightly over. The 25% retry buffer exists for this. The difference between the two main-arm choices is too small for cost to decide between them, so quality must.

The storage line is where the arms differ most in relative terms: a LoRA checkpoint is an adapter plus its optimizer state, while a full or A2 checkpoint carries the model and its optimizer. At three checkpoints for two months this is still $1.6 against $17.3.

7. Scope: what this does not measure

This note measures **fit, throughput and cost**. It does not measure whether any arm teaches the model Filipino, how much each arm forgets, or which learning rate each arm needs. Those questions belong to the pilot head-to-head, in which the three arms train on the same real Filipino data (100M tokens each) and are scored on held-out evaluation. The losses in this note come from 25 steps on random tokens and are not evidence either way.

8. Reproducibility

Table 11: Result files (experiments/results/gate-f/modal/), their device strings, CUDA runtime and driver where recorded, and the loss-equality check (the same fixed sequence in every attempt of a file).

arm · GPU	file (google-gemma-4-e4b.*)	device string	CUDA / driver	loss check abs diff
A1 · A100	`v4-a1.json`	A100-SXM4-80GB	13.0 / 580.95.05	6.37e-04
A1 · A100 (sensitivity)	`v4-a1.pcie-1.json`	A100 80GB PCIe	not recorded	6.37e-04
A1 · H100	`v4-a1-h100.json`	H100 80GB HBM3	13.0 / 580.95.05	3.62e-04
A2 · A100	`v4-a2.json`	A100-SXM4-80GB	13.0 / 580.95.05	6.37e-04
A3 · A100	`v4-a3.json`	A100-SXM4-80GB	not recorded	6.37e-04
A3 · H100	`v4-a3-h100.json`	H100 80GB HBM3	13.0 / 580.95.05	3.62e-04
A2 (crashed run 1)	`v4-a2.run1-crash-tiecheck.json`	not recorded	not recorded	exit 1 at mb 1 (ckpt off and on)

Table 12: Files.

item	path
probe (host-agnostic)	`scripts/training/gate_f_throughput.py` (`--arm lora`, `lora_emb`, `full`; `--ckpt auto`)
launcher (A100 and H100 functions, fast image + torchao)	`scripts/training/modal_gate_f.py`
all-in cost script and its output	`experiments/results/gate-f/v4/allin_cost.py`, `allin_cost.txt`
container log	`experiments/results/gate-f-modal-v4.log`
tables and figure of this note	`experiments/reports/_eng003.py`

`_eng003.py` checks every file against the registered method (arm, sequence length, steps, search order, loss check present, finite losses, tokens/s = micro-batch × 4,096 / seconds per step). It also reparses `allin_cost.txt`, recomputes every line from the JSONs and the constants declared in `allin_cost.py`, and asserts agreement. If an A1 file recorded on `A100-SXM4-80GB` is present, the script uses it as the registered A1-A100 row and keeps the PCIe file as a sensitivity row.

9. Limitations

- **One measurement per cell.** Every tokens/s value is a single 20-step run. ENG-001′s only repeat moved by up to 4.6%, and we apply about ±4% to every number here. The 4% gap between A1 and A3 at their selected settings is within it. The 4.6% A100-SXM4 gap is at its edge. The 10% (H100) and 12.7% (A100) equal-settings gaps are outside it.
- **Synthetic tokens.** Step time does not depend on token content at fixed length without packing, but losses do. No loss in this note says anything about learning.
- **The GPU variant cannot be selected.** A1′s first A100 run landed on PCIe and needed a second run to get the registered SXM4 row. A2 has no H100 number, so its all-in cost is A100-only.
- **File-level flags.** A result file stores `method_flags` from its last successful attempt, so `gradient_checkpointing: true` at file level does not describe a selected checkpointing-off attempt. The per-attempt `ckpt` field is the authoritative one and is what this note uses.
- **Lower-bound hours.** Data loading, checkpoint writes, evaluation pauses and restarts are excluded. The 25% buffer is meant to cover them, and is not a measurement of them.
- **Estimates in the all-in cost.** Evaluation at 10 A100-hours and the checkpoint sizes are declared estimates. The plumbing test is priced at the upper end (50M tokens) of its planned 20–50M.
- **One learning rate per arm, not tuned.** The probe's learning rates only keep the optimizer honest. The pilot has to set them per arm.
- **Single-GPU only.** Multi-GPU and other GPU types were not measured.

Acknowledgments

We thank the leadership of Kooya PH for supporting this work, and the native-speaker evaluators who reviewed and rated the Filipino and Taglish material.

AI assistance. Analysis code, statistical checks and drafts of this document were prepared with the help of Claude (Anthropic), an AI model. Every number is generated from the project's result files. The author reviewed the content and takes responsibility for it.